

AI-Based Automated Travel Planner

Detailed Project Report, Technical Documentation, and Result Record

This report presents the complete explanation of the AI-Based Automated Travel Planner m project. The application is designed as an India tourist guide that accepts user goals, budg travel type, days, departure city, transport preference, hotel preference, and multiple selec destinations. It then builds an intelligent, feasible, and optimized route plan using classical AI planning concepts and a domain-specific travel knowledge base.

Document coverage

- Problem statement and project objectives.
- Detailed explanation of STRIPS, Goal Stack Planning, and Knowledge-Based Planning.
- System architecture and flow of control.
- Routing calculations, time estimation, and budget estimation formulas.
- Technologies, frameworks, and libraries used.
- Step-by-step working of the application.
- Manual solved example using sample input values.
- Output screenshots and generated results.
- How to run the project in VS Code.

Problem Statement and Objectives

Problem statement: design an AI planner that generates optimal travel plans based on use constraints, and available actions. In this project the problem is specialized to India tourism. Users may select multiple destinations, and the system must construct a practical circular travel plan from a departure city, identify route stopovers, estimate cost and time, and generate a complete itinerary.

Main objectives

- Accept multiple destinations instead of a single city.
- Support real trip constraints such as days, budget, transport, and hotel type.
- Find the best route ordering using estimated distance, time, and budget
- Show what to visit, what food to try, and what experiences to achieve.
- Generate a day-wise schedule that looks like a tourist guide.
- Demonstrate AI planning concepts in both code form and PDDL form.

Expected outcome

The final system is a local web application that opens on a local IP address, accepts trip parameters, computes the optimized multi-city route, displays the planner actions, prints segment costs and travel times, and shows a complete itinerary together with attractions, and experiences.

Core Concepts Used

STRIPS

STRIPS stands for Stanford Research Institute Problem Solver. It is a classical planning representation where every action is defined through three logical parts: preconditions, add effects, and delete effects. Preconditions describe what must already be true before an action can execute. Add effects describe what becomes true after the action. Delete effects describe what becomes false after the action. In this project, the actions are represented through the Action class in `planner/goal_stack.py`. For example, `reserve_transport` can be executed only after `destination_selected` is true and the user's preferred transport is available in the current state. Once executed, it adds `transport_reserved` to the state.

Goal Stack Planning

Goal Stack Planning is a backward reasoning planning method. Instead of trying random actions, the planner starts from the desired goals and asks which action can satisfy each goal. It pushes missing goals on a stack, chooses an action whose add effects contain the goal, and then pushes the action's preconditions. When all preconditions are satisfied, the action is applied and the state is updated. This continues until all goals are satisfied. In the project,

`GoalStackPlanner.plan` follows exactly this mechanism and creates the final sequence `choose_destination`, `book_beach_activity`, `reserve_transport`, `reserve_hotel`, `prepare_itinerary`, and `confirm_trip`.

Knowledge-Based Planning

Knowledge-based planning means the planner does not work only with abstract actions; it also uses a domain knowledge base. Here the knowledge base contains destination coordinates, hotel areas, attractions, route corridors, stopover details, featured experiences, and food lists.

This knowledge allows the planner to produce meaningful tourist guidance, not just symbolic plans. Because of this layer, the system behaves like a practical travel guide rather than a generic AI planner.

Technologies, Frameworks, and Libraries

Python

Python is the main programming language. It is used for routing formulas, planning logic, data structures, the local web server, and report generation. Python was chosen because it is concise, expressive, and appropriate for AI and algorithmic demonstrations.

PDDL

PDDL is the Planning Domain Definition Language. The project exports domain.pddl and problem.pddl to show how the travel planning problem can be represented in formal AI planning syntax. This is useful for academic explanation and planning simulator compatibility.

HTTP Server

The web interface is built with Python's built-in http.server module. This keeps the project concise and avoids unnecessary external frameworks while still providing a working local website.

urllib.parse

The parse_qs function is used to decode submitted form values from the HTML form.

itertools

The permutations function is used to enumerate all possible orderings of selected destinations so the system can compare route scores.

math

The math module supports the haversine distance computation for geographic estimation.

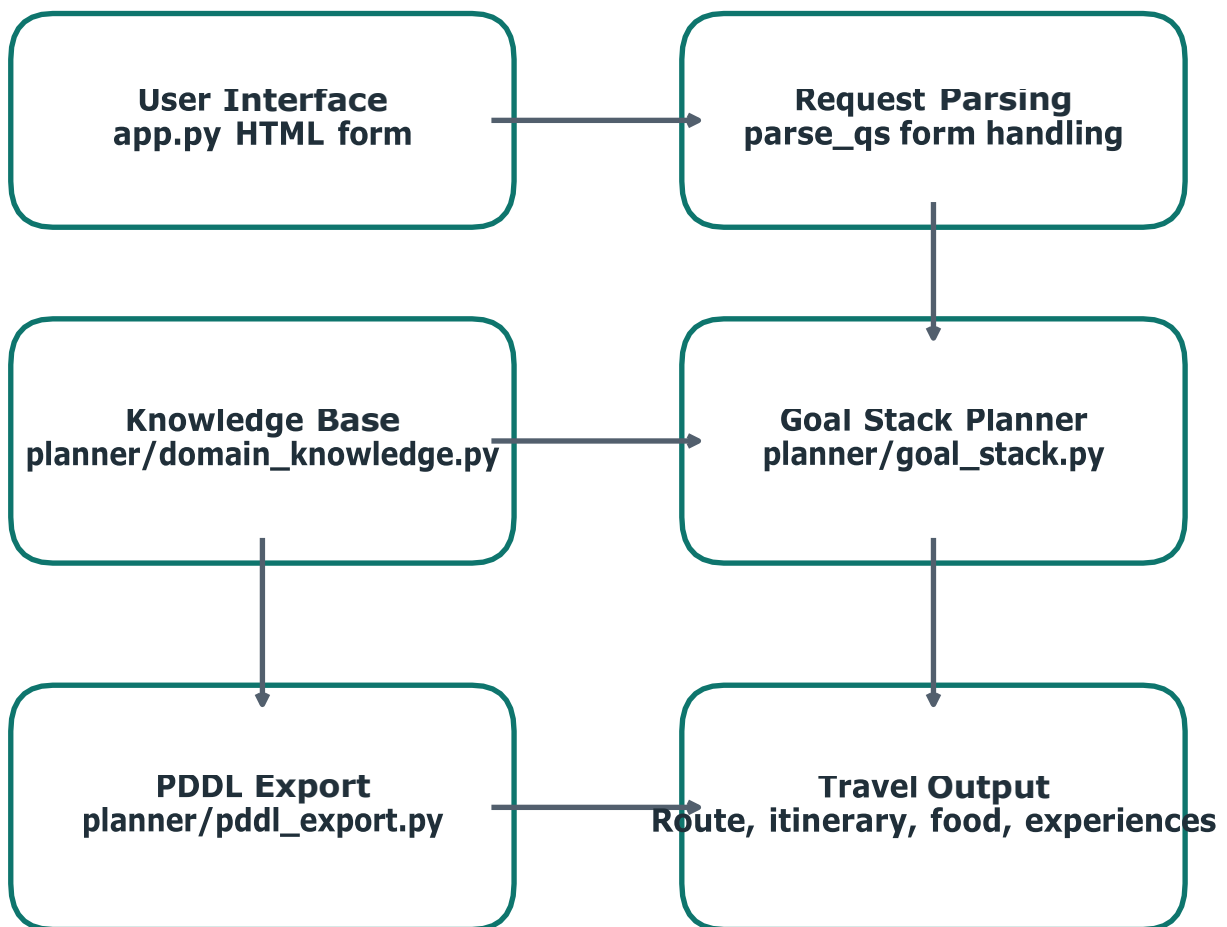
pathlib

pathlib is used to create and write PDDL files and documentation files in a clean way.

matplotlib and PIL

These libraries are used in the documentation generator to create the PDF, result visuals, and screenshot-style output images.

System Architecture



Architecture flow: the user submits travel constraints through the website. The request parser extracts selected destinations and other values. The knowledge module computes route feasibility tourist facts. Goal Stack Planning builds a valid action order. PDDL exporter serializes the planning problem. The final HTML response renders a professional tourist guide result.

Detailed Working of the Application

- Step 1: The user opens the local website and selects one or more destinations, travel goal, budget, number of days, departure city, transport, and hotel type.
- Step 2: app.py receives the POST request and converts HTML form data into a Python dictionary.
- Step 3: build_problem_data in planner/domain_knowledge.py reads user constraints and selects valid destinations.
- Step 4: optimize_route checks every permutation of the selected destinations and computes route score using distance, time, and budget.
- Step 5: build_itinerary creates day-wise travel and exploration activities, including route stops and main destinations.
- Step 6: GoalStackPlanner creates the formal action sequence required to satisfy the goal state.
- Step 7: pddl_export.py converts the same planning instance into domain.pddl and problem.pddl.
- Step 8: app.py renders the final page showing route segments, itinerary, places, food, experiences, actions, and PDDL output.

Important project modules

app.py handles the website. planner/domain_knowledge.py stores travel facts and route planner/goal_stack.py implements STRIPS-style planning. planner/pddl_export.py writes the formal planning files.

System Architecture Explanation

The system architecture follows a simple but effective pipeline. The browser acts as the presentation layer. The Python HTTP server acts as the controller layer. The knowledge and planning modules form the business logic layer. Finally, the PDDL exporter and HTML render form the output layer. The architecture is intentionally lightweight so that the project remains concise, easy to run, and easy to explain in an academic setting.

Architecture components

- Presentation layer: HTML generated by app.py.
- Input layer: POST form parsing with parse_qs.
- Knowledge layer: destination facts, stopovers, hotel areas, and route corridors.
- Planning layer: Action representation and Goal Stack Planning algorithm.
- Optimization layer: route permutation evaluation with score calculation.
- Export layer: PDDL file generation for domain and problem description.
- Output layer: itinerary, route summary, guide cards, and planning explanation.

This modular design makes the project maintainable. For example, new destinations can be added by extending the knowledge base only, without rewriting the planner. Similarly, a different optimizer can be attached later without changing the core route optimization logic.

Routing Calculations and Optimization

The optimization module estimates route quality using three measures: total distance, total time, and total budget. Because the app is offline and does not call live map services, the project uses coordinate-based estimation rather than exact road-network distances. This is appropriate for a mini project because it demonstrates the AI reasoning pipeline while keeping the code concise.

Formula summary

- Base geographic distance is computed using the haversine formula on latitude and longitude coordinates.
- Road adjusted distance = $\text{round}(\text{haversine_distance} * 1.18)$.
- Time for one segment = $\text{distance} / \text{transport_speed} + \text{stopover_penalty}$.
- Stopover penalty = $\text{number_of_stopovers} * 1.4 \text{ hours}$.
- Budget for one segment = $\text{distance} * \text{cost_per_km} + \text{fixed_charge}$.
- Final route score = $\text{total_distance} + \text{total_time} * 12 + \text{total_budget} / 140$.

Why the score works

The score combines the three criteria into one comparable value. A lower score means the route is shorter, faster, and cheaper overall. The coefficients are chosen so that time and budget influence the decision without completely dominating physical distance. This gives a balanced educational example of multi-factor route evaluation.

Worked Manual Example

Manual sample input

- Departure city: Chennai
- Selected destinations: Goa and Jaipur
- Goal: Culture
- Budget: Medium
- Days: 7
- Transport: Train
- Hotel type: Standard

Possible route orders

Because there are two selected destinations, the planner evaluates two permutations: Chennai -> Goa -> Jaipur -> Chennai and Chennai -> Jaipur -> Goa -> Chennai. For each route it computes segment distance, time, and budget, then sums them and calculates the final score. The route with the lower score becomes the chosen route.

Planner interpretation

For the current sample data, the computed route is Chennai -> Goa -> Jaipur -> Chennai. The Stack Planner then derives the action order: choose_destination -> book_beach_activity -> reserve_transport -> reserve_hotel -> prepare_itinerary -> confirm_trip. This means the system first confirms a valid destination selection, then books the featured activity, reserves transport, reserves hotel, prepares the itinerary, and finally confirms the trip.

Manual Example Calculations

Segment 1: Chennai -> Goa

Estimated distance = 834 km. Estimated time = distance / train_speed + stopover_penalty
=
17.2 hours. Estimated budget = 1568 rupees. Stopovers on this segment: Bengaluru, Hampi.

Segment 2: Goa -> Jaipur

Estimated distance = 1537 km. Estimated time = distance / train_speed + stopover_penalty
=
29.3 hours. Estimated budget = 2552 rupees. Stopovers on this segment: Mumbai, Udaipur.

Segment 3: Jaipur -> Chennai

Estimated distance = 1896 km. Estimated time = distance / train_speed + stopover_penalty
=
35.5 hours. Estimated budget = 3054 rupees. Stopovers on this segment: Hyderabad, Bhopal.

Final totals: total distance = 4267 km, total time = 82.0 hours, total budget = Rs

PDDL Representation

PDDL is included so the project can be explained as a formal AI planning system rather than a custom algorithm. The exported files show the predicates, action definitions, initial state, and goal state of the current travel request.

Domain highlights

```
(define (domain travel-planner) (:requirements :strips)
  (:predicates (user_ready) (want_multi_city_route)
               (want_goa)      (want_jaipur))
```

```

(goal_culture)    (prefer_train)    (hotel_standard)
(depart_chennai) (destination_selected)
                (beach_activity_booked)
                (transport_reserved) (hotel_reserved)
                (itinerary_ready)  (trip_confirmed) )
                (:action choose_destination :precondition
(and (user_ready) (want_multi_city_route))
:effect (and (destination_selected)) ) (:action book_beach_activity
:precondition (and (destination_selected) (goal_culture)) :effect (and (beach_activity_booked)) ) (:action reserve_transport
prepare_itinerary :precondition (and
(beach_activity_booked) (transport_reserved)
(hotel_reserved)) :effect (and (itinerary_ready)) )
(:ac

```

Problem highlights

```

(define (problem india-tour-plan) (:domain travel-
planner) (:init (user_ready) (want_multi_city_route)
(want_goa) (want_jaipur) (goal_culture)
(prefer_train)    (hotel_standard)
(depart_chennai) ) (:goal (and
(destination_selected)
(beach_activity_booked)
(transport_reserved)(hotel_reserved)
(itinerary_ready) (trip_confirmed) ) ) )

```

Result Screenshots and Output

The following images are generated from the sample output of the application

127.0.0.1:8000

India Tourist Guide Planner

Select multiple destinations and the planner will produce the best circular route from Chennai with sightseeing stops, itinerary, food, and experiences.

PythonPDDLSTRIPSGoal Stack Planning

Choose One Or More Main Destinations

☒ Goa
Beach

☒ Jaipur
Heritage

☐ Kerala
Nature

☐ Manali
Mountain

☒ Varanasi
Spiritual

Travel Goal

relaxation

Budget

medium

Days

6

Departure City

Chennai

Transport

train

Hotel Type

standard

Generate Professional Travel Plan

Professional Tourist Guide Plan

Optimized Route: Chennai -> Goa -> Jaipur -> Varanasi -> Chennai

Total Distance: 4883 km

Total Time: 95.4 hours

Estimated Budget: Rs. 8437

127.0.0.1:8000

Professional Tourist Guide Plan

Optimized Route: Chennai -> Goa -> Jaipur -> Varanasi -> Chennai

Total Distance: 4883 km

Total Time: 95.4 hours

Estimated Budget: Rs. 8437

Featured Activity: Baga Beach water sports session

Suggested First Hotel Area: Calangute

Planner Actions

1. choose_destination

2. book_beach_activity

3. reserve_hotel

4. reserve_transport

5. prepare_itinerary

6. confirm_trip

Best Route Segments

Chennai to Goa

Distance: 834 km

Time: 17.2 hours

Budget: Rs. 1568

Route Stops: Bengaluru, Hampi

Goa to Jaipur

Distance: 1537 km

Time: 29.3 hours

Budget: Rs. 2552

Route Stops: Mumbai, Udaipur

Jaipur to Varanasi

Distance: 872 km

Time: 17.8 hours

Budget: Rs. 1621

Route Stops: Agra, Prayagraj

Varanasi to Chennai

Distance: 1640 km

Time: 31.1 hours

Budget: Rs. 2696

Route Stops: Vijayawada, Hyderabad

Day-Wise Itinerary

Day-Wise Itinerary

Day 1: Travel: Chennai to Goa

Morning: Start from Chennai and follow the optimized route: Chennai -> Bengaluru -> Hampi -> Goa.

Afternoon: Stop at Bengaluru, Hampi

Evening: Reach Goa after about 17.2 hours covering 834 km.

Must Taste: masala dosa

Day 2: Explore Goa

Morning: Visit Baga Beach.

Afternoon: Taste Goan fish curry and continue to Fort Aguada.

Evening: Enjoy sunset beach walk.

Must Taste: Goan fish curry

Day 3: Explore Goa

Morning: Visit Fort Aguada.

Afternoon: Taste bebinca and continue to Fontainhas.

Evening: Enjoy water sports.

Must Taste: bebinca

Day 4: Travel: Goa to Jaipur

Morning: Start from Goa and follow the optimized route: Goa -> Mumbai -> Udaipur -> Jaipur.

Afternoon: Stop at Mumbai, Udaipur

Evening: Reach Jaipur after about 29.3 hours covering 1537 km.

Must Taste: vada pav

Day 5: Explore Jaipur

Morning: Visit Amber Fort.

Afternoon: Taste dal baati churma and continue to City Palace.

Evening: Enjoy folk dance evening.

Must Taste: dal baati churma

Day 6: Explore Jaipur

Morning: Visit City Palace.

Afternoon: Taste pyaaz kachori and continue to Hawa Mahal.

Evening: Enjoy bazaar shopping.

Must Taste: pyaaz kachori

Places To Visit Across Main Destinations And Route Stops

Places To Visit Across Main Destinations And Route Stops

Goa Type: Main Destination Places: Baga Beach, Fort Aguada, Fontainhas, Dudhsagar Falls Food: Goan fish curry, bebinca, prawn balchao, poi bread Experiences: sunset beach walk, water sports, night market visit, Latin quarter heritage stroll	Jaipur Type: Main Destination Places: Amber Fort, City Palace, Hawa Mahal, Jantar Mantar Food: dal baati churma, pyaaz kachori, ghevar, laal maas Experiences: folk dance evening, bazaar shopping, fort light show, block print shopping	Varanasi Type: Main Destination Places: Kashi Vishwanath Temple, Dashashwamedh Ghat, Sarnath, Assi Ghat Food: kachori sabzi, malaiyyo, tamatar chaat, banarasi paan Experiences: Ganga aarti, sunrise boat ride, temple trail, silk weaving lane walk	Bengaluru Type: Route Stop Places: Cubbon Park, Bangalore Palace Food: masala dosa Experiences: brewery street evening
Hampi Type: Route Stop Places: Virupaksha Temple, Stone Chariot Food: north Karnataka meals Experiences: sunset at Hemakuta Hill	Mumbai Type: Route Stop Places: Gateway of India, Marine Drive Food: vada pav Experiences: sunset at Marine Drive	Udaipur Type: Route Stop Places: City Palace, Lake Pichola Food: dal bati Experiences: boat ride	Agra Type: Route Stop Places: Taj Mahal, Agra Fort Food: petha Experiences: sunrise monument view
Prayagraj Type: Route Stop Places: Triveni Sangam, Anand Bhavan Food: kachori Experiences: ghat boat stop	Vijayawada Type: Route Stop Places: Kanaka Durga Temple, Bhavani Island Food: Andhra meals Experiences: riverfront stop	Hyderabad Type: Route Stop Places: Charminar, Golconda Fort Food: Hyderabadi biryani Experiences: night market visit	

Planning Model Used

- **STRIPS:** Planning actions use preconditions and effects.
- **Goal Stack Planning:** The planner creates the required action sequence.
- **Knowledge-Based Planning:** India travel facts drive routing and sightseeing advice.

PDDL Output

```
(define (domain travel-planner)
  (:requirements :strips)
  (:predicates
    (user_ready)
    (want_multi_city_route)
    (want_goa)
    (want_jaipur)
    (want_varanasi)
    (goal_relaxation)
    (prefer_train)
    (hotel_standard)
    (depart_chennai)
    (destination_selected)
    (beach_activity_booked)
    (transport_reserved)
    (hotel_reserved)
    (itinerary_ready)
    (trip_confirmed)
  )

  (:action choose_destination
    :precondition (and (user_ready) (want_multi_city_route))
    :effect (and (destination_selected))
  )

  (:action book_beach_activity
    :precondition (and (destination_selected) (goal_relaxation))
    :effect (and (beach_activity_booked))
  )

  (:action reserve_transport
    :precondition (and (destination_selected) (prefer_train) (depart_chennai))
    :effect (and (transport_reserved))
  )

  (:action reserve_hotel
    :precondition (and (destination_selected) (hotel_standard))
    :effect (and (hotel_reserved))
  )

  (:action prepare_itinerary
    :precondition (and (beach_activity_booked) (transport_reserved) (hotel_reserved))
    :effect (and (itinerary_ready))
  )

  (:action confirm_trip
    :precondition (and (itinerary_ready))
    :effect (and (trip_confirmed))
  )
)
```

```
(define (problem india-tour-plan)
  (:domain travel-planner)
  (:init
    (user_ready)
    (want_multi_city_route)
    (want_goa)
    (want_jaipur)
    (want_varanasi)
    (goal_relaxation)
    (prefer_train)
    (hotel_standard)
    (depart_chennai)
  )
  (:goal
    (and
      (destination_selected)
      (beach_activity_booked)
      (transport_reserved)
      (hotel_reserved)
      (itinerary_ready)
      (trip_confirmed)
    )
  )
)
```

Another example result analysis:



Sample Day-Wise Itinerary

Day	Title	Morning Plan	Must Taste
Day 1	Travel: Chennai to Goa	Start from Chennai and follow the optimized route: Chennai -> Bengaluru -> Hampi -> Goa	masala dosa
Day 2	Explore Goa	Visit Baga Beach.	Goan fish curry
Day 3	Explore Goa	Visit Fort Aguada.	bebinca
Day 4	Explore Goa	Visit Fontainhas.	prawn balchao
Day 5	Travel: Goa to Jaipur	Start from Goa and follow the optimized route: Goa -> Mumbai -> Udaipur -> Jaipur	vada pav
Day 6	Explore Jaipur	Visit Amber Fort.	dal baati churma

The sample result shows the optimized route Chennai -> Goa -> Jaipur -> Chennai with total distance 4267 km, total estimated time 82.0 hours, and total budget Rs. 7174. The itinerary image shows that the system not only chooses where to go, but also structures how each day can be spent.

How to Run the Project

Software requirements

- Python installed on the system.
- VS Code or any Python-capable editor.
- A local browser such as Chrome or Edge.
-

Execution steps

- Open the project folder in VS Code.
- Open the terminal in the same folder.
- Run the command `python app.py`
- Open the address `http://127.0.0.1:8000` in the browser.
- Select one or more destinations and other travel constraints.
- Click Generate Professional Travel Plan to see the result.
- The files `pddl/domain.pddl` and `pddl/problem.pddl` are also generated or updated.

To regenerate this report later, run `python docs\generate_report.py` from the project root.

Conclusion and Future Scope

This project demonstrates that classical AI planning can be combined with practical tourist guidance in a compact Python application. The system not only proves the use of STRIPS a Stack Planning, but also extends them using a knowledge base to create human-friendly tr recommendations. The result is academically meaningful and practically understandable.

Strengths

- Clear mapping from planning theory to working code.
- Multi-destination support with route optimization.
- Feasible, readable itinerary generation.
- Academic PDDL export for planning explanation.
- Lightweight local deployment.

Future improvements

- Use real map APIs for live traffic-aware routes.
- Add hotel pricing APIs and real booking integrations.
- Store user profiles and saved trip history.
- Add downloadable PDF itineraries from the app itself.
- Release a mobile application with backend database support